# PR Outreach Intelligence — Testing Guide

**Version:** 2.0
**Date:** April 2026
**Stack:** Express 5 · Vue 3 · Vitest · Playwright · PostgreSQL · pnpm monorepo
**Total Tests:** 376 (254 backend + 110 frontend + 12 E2E)
**Test Files:** 45
**Status:** All passing ✓ — CI/CD pipeline active

---

## 1. Why We Test (And Why It Matters Here)

PR Outreach Intelligence isn't a todo app. It handles **authentication with token rotation**, **transactional email sending**, **AI-generated content**, and **multi-table PostgreSQL transactions**. A single broken query can corrupt relationship scores across contacts, campaigns, and interactions.

Testing this project serves three purposes:

1. **Catch regressions before they reach production.** When you change the interaction scoring logic, you need to know immediately if campaigns break.
2. **Document behavior as executable specs.** Reading `auth.service.test.ts` tells you more about token rotation than any wiki page.
3. **Enable confident refactoring.** The codebase uses raw SQL everywhere (no ORM safety net). Tests are the safety net.

> The rule of thumb: if it touches the database, handles money, or manages auth, it gets tested. No exceptions.

---

## 2. The Stack at a Glance

| Layer | Technology | Test Tool |
|---|---|---|
| Backend runtime | Express 5.2 + TypeScript 6 | Vitest 4.1 (node environment) |
| Database | PostgreSQL 15 (raw `pg`, no ORM) | Real test DB ( `pr_ai_db_test` ) |

| | | |
|---|---|---|
| Validation | Zod 4.3 | Unit tests (schema `.safeParse()`) |
| Auth | JWT + bcryptjs + SHA-256 refresh tokens | Integration tests against real DB |
| API routes | Express 5 controllers + middleware chain | supertest 7.2 (HTTP integration) |
| AI providers | OpenAI / Gemini via native fetch | Mocked with `vi.mock()` |
| Email | Resend / Console provider | Mocked with `vi.mock()` |
| Frontend framework | Vue 3.5 + Pinia 3.0 + Vue Router 5 | Vitest 4.1 (jsdom environment) |
| Frontend HTTP | Native fetch (no Axios) | `vi.fn()` stubs |
| Component testing | @vue/test-utils 2.4 | jsdom |
| E2E testing | Playwright 1.59 | Chromium |
| Monorepo | pnpm workspaces | Shared `@pr-outreach/shared-types` |
| CI/CD | GitHub Actions | 5 parallel jobs |

---

## 3. Project Structure (Test Files)

```
backend/src/
├── __tests__/
│   ├── setup.ts                ← env bootstrap
│   └── helpers.ts              ← cleanDatabase(), createTestUser(), authToken()
├── shared/
│   ├── errors/__tests__/       ← AppError.test.ts (9)
│   ├── utils/__tests__/        ← utils.test.ts (12)
│   └── middlewares/__tests__/  ← authenticate (7), validate (5), errorHandler (4)
└── modules/
    ├── auth/__tests__/         ← dto (16), service (13), api (17)
    ├── contacts/__tests__/     ← dto (11), service (13), api (13)
    ├── campaigns/__tests__/    ← dto (12), service (14), api (12)
    ├── interactions/__tests__/ ← dto (9), service (10), api (9)
    ├── dashboard/__tests__/    ← service (5), api (6)
    ├── ai/__tests__/           ← dto (9), prompt (10), service (8), api (12)
    └── messaging/__tests__/    ← dto (6), service (6), api (6)
```

```
frontend/src/
├── __tests__/setup.ts
├── api/__tests__/              ← client.test.ts (9), client.refresh.test.ts (3)
├── utils/__tests__/            ← date.test.ts (6)
├── composables/__tests__/      ← useNotifications.test.ts (5)
├── stores/__tests__/           ← auth (7), contacts (8), campaigns (11),
│                                  interactions (6)
├── router/__tests__/           ← router.test.ts (6)
├── views/auth/__tests__/       ← LoginView (7), RegisterView (7)
├── views/outreach/__tests__/   ← OutreachDraftView (10)
├── components/contacts/__tests__/     ← ContactFormModal (8)
├── components/campaigns/__tests__/    ← CampaignFormModal (8)
└── components/interactions/__tests__/ ← InteractionForm (9)

e2e/
├── helpers.ts                  ← freshUser(), registerViaUI/API(), injectAuth()
├── auth.spec.ts                ← 5 tests
├── contacts.spec.ts            ← 2 tests
├── campaigns.spec.ts           ← 2 tests
└── outreach.spec.ts            ← 3 tests

.github/workflows/
└── ci.yml                      ← 5-job CI pipeline
```

# 4. Infrastructure Setup

## 4.1 Backend Vitest Configuration

```
// backend/vitest.config.ts
export default defineConfig({
  test: {
    globals: true,          // no need to import describe/it/expect
    environment: "node",    // backend = Node, not browser
    root: "src",            // resolve from src/ so imports match app code
    setupFiles: ["./__tests__/setup.ts"],
    include: ["**/__tests__/**/*.test.ts"],
    testTimeout: 15_000,    // DB ops can be slow on first run
    hookTimeout: 15_000,
    pool: "forks",          // isolate test files in child processes
    fileParallelism: false, // ← critical: prevents DB deadlocks
  },
});
```

**Why `fileParallelism: false`?** All integration tests share the same PostgreSQL database. Running them in parallel causes deadlocks on `TRUNCATE CASCADE` and foreign key violations. Sequential file execution solves this while still allowing tests *within* a file to run fast.

**Why `pool: "forks"`?** Vitest's default thread pool shares memory, which causes issues with `pg` connection pools. Forked processes get their own connection pool instance.

## 4.2 Setup File

```
// backend/src/__tests__/setup.ts
process.env.NODE_ENV = "test";
import dotenv from "dotenv";
dotenv.config({ path: path.resolve(process.cwd(), ".env.test"), override: true }
```

This runs before every test file. It loads `.env.test` which points to `pr_ai_db_test` on port 5433 — a dedicated test database that gets truncated between suites.

## 4.3 Test Helpers

```
// cleanDatabase() — resets all 8 tables between tests
await pool.query(`
  TRUNCATE users, auth_sessions, contacts, campaigns,
           campaign_contacts, interactions, ai_suggestions, audit_events
  CASCADE
`);

// createTestUser() — inserts a user directly (bypasses service layer)
// Uses bcrypt with 4 rounds (fast) instead of production's 12
const hash = await bcrypt.hash(password, 4);

// authToken() — generates a valid JWT for HTTP tests
jwt.sign({ userId, email }, env.JWT_SECRET, { expiresIn: "5m" });
```

**Why bypass the service layer for user creation?** Service tests need a user to exist *before* the test starts. Using the auth service would test two things at once. The helper inserts directly via SQL, keeping test setup independent of the code being tested.

### 4.4 Frontend Vitest Configuration

```
// frontend/vitest.config.ts
export default defineConfig({
  plugins: [vue()],
  resolve: { alias: { '@': './src' } },
  test: {
    globals: true,
    environment: 'jsdom',    // simulates browser DOM
    include: ['src/**/__tests__/**/*.test.ts'],
    setupFiles: ['src/__tests__/setup.ts'],
  },
});
```

Frontend tests use `jsdom` to simulate browser APIs (DOM, localStorage, fetch).

### 4.5 Playwright Configuration

```
// playwright.config.ts
export default defineConfig({
  testDir: './e2e',
  fullyParallel: false,
```

```
  workers: 1,
  timeout: 30_000,
  use: {
    baseURL: 'http://localhost:5173',
    trace: 'on-first-retry',
    screenshot: 'only-on-failure',
  },
  projects: [{ name: 'chromium', use: { ...devices['Desktop Chrome'] } }],
  webServer: [
    { command: 'pnpm --filter @pr-outreach/backend dev',
      url: 'http://localhost:3000/health' },
    { command: 'pnpm --filter @pr-outreach/frontend dev',
      url: 'http://localhost:5173' },
  ],
});
```

E2E tests auto-start both servers (or reuse existing ones). The `webServer` config waits for the health endpoint and frontend to be available before running tests.

# 5. Testing Patterns & Conventions

## 5.1 The Test Categories

| Category | DB? | Mocks? | Speed | Example |
|---|---|---|---|---|
| **DTO / Schema** | No | No | ~20ms | `contacts.dto.test.ts` |
| **Service Integration** | Yes (real) | External APIs only | ~300ms/test | `contacts.service.test.ts` |
| **API Integration** | Yes (real) | External APIs only | ~400ms/test | `contacts.api.test.ts` |
| **Middleware Unit** | No | Express req/res | ~30ms | `authenticate.test.ts` |
| **Frontend Unit** | No | API layer mocked | ~50ms | `campaigns.store.test.ts` |
| **Component** | No | Stores/APIs mocked | ~80ms | `ContactFormModal.test.ts` |
| **E2E Smoke** | Yes (real) | Nothing | ~2s/test | `auth.spec.ts` |

## 5.2 Service Integration Test Lifecycle

Every service test file follows this exact pattern:

```
beforeAll(async () => {
  const schema = fs.readFileSync("../config/schema.sql", "utf-8");
  await pool.query(schema);
});

afterAll(() => pool.end());

beforeEach(async () => {
  await cleanDatabase();
  const user = await createTestUser();
  userId = user.id;
});
```

### 5.3 API Integration Test Lifecycle

API tests use `supertest` to send real HTTP requests through the full Express middleware chain:

```
import request from "supertest";
import { app } from "../../__tests__/helpers";

beforeEach(async () => {
  await cleanDatabase();
  const user = await createTestUser();
  token = authToken(user.id, user.email);
});

it("should create a contact (201)", async () => {
  const res = await request(app)
    .post("/api/contacts")
    .set("Authorization", `Bearer ${token}`)
    .send({ name: "Alice", email: "alice@press.com", outlet: "Reuters" });

  expect(res.status).toBe(201);
  expect(res.body.name).toBe("Alice");
});
```

**Why both service and API tests?** Service tests verify business logic in isolation. API tests verify the full HTTP cycle: routing, middleware chain (validation, auth, rate limiting), controller parameter parsing, and response formatting. A service test passing doesn't guarantee the controller wires things correctly.

### 5.4 Component Test Pattern

Vue components are tested with `@vue/test-utils`. Modals using `watch` on an `open` prop require mounting with `open: false` and then calling `setProps({ open: true })` to trigger the watcher:

```
function mountModal(props = {}) {
  HTMLDialogElement.prototype.showModal = vi.fn();
  HTMLDialogElement.prototype.close = vi.fn();
  return mount(ContactFormModal, {
    props: { open: false, contact: null, ...props },
    global: { plugins: [createPinia()] },
  });
}
```

```
it('calls store.create on submit', async () => {
  const wrapper = mountModal({ open: false });
  await wrapper.setProps({ open: true }); // triggers watch
  // ... fill inputs, submit form
});
```

### 5.5 Mocking External Dependencies

For AI and email providers, we mock at the module level:

```
vi.mock("../ai.provider", () => ({
  generateCompletion: vi.fn().mockResolvedValue({
    subject: "Test subject", body: "Test body",
  }),
  getActiveModel: vi.fn().mockReturnValue("mock-model"),
}));
```

**The rule:** We mock *external boundaries* (OpenAI, Resend, etc.) but never mock the database. The database is part of the system under test.

### 5.6 Error Assertion Strategy

Due to a quirk in how `AppError` uses `Object.setPrototypeOf`, `instanceof` checks for subclasses break under Vitest's module isolation. We assert on the error message string:

```
// ✅ Reliable across all environments
await expect(fn()).rejects.toThrow("Contact not found");
```

### 5.7 E2E Test Pattern

E2E tests use unique users per test to avoid data collision:

```
test('create a contact and see it in the list', async ({ page, baseURL }) => {
  const user = freshUser();                        // unique email per test
  const tokens = await registerViaAPI(baseURL!, user); // fast API setup
  await injectAuth(page, tokens);                  // inject refresh token

  await page.goto('/contacts');
```

```
  // ... interact with real UI, real backend, real database
});
```

# 6. Backend Tests — Module Breakdown

## 6.1 Shared Layer (37 tests)

### AppError Hierarchy (9 tests)

Tests all error subclasses: `BadRequestError(400)`, `UnauthorizedError(401)`, `ForbiddenError(403)`, `NotFoundError(404)`, `ConflictError(409)`. Verifies status codes, messages, and that the base `AppError` class works correctly.

### Utility Functions (12 tests)

- **`parseId`**: valid integers, zero, negatives, non-numeric strings, floats, arrays
- **`parsePagination`**: defaults (limit=50, offset=0), valid values, cap at 200, negatives

### Validate Middleware (5 tests)

Tests the Zod validation middleware with valid body, missing fields, invalid email, error message formatting, and empty body.

### Authenticate Middleware (7 tests)

Mocks `JWT_SECRET` via `vi.mock("config/env")`. Tests: valid JWT extraction → `req.userId`, missing Authorization header, non-Bearer scheme, expired token, wrong secret, malformed token, empty payload.

### Error Handler Middleware (4 tests)

Tests the Express error handler: known errors (BadRequest → 400), NotFound → 404, unknown errors → 500, and custom AppError subclasses.

## 6.2 Auth Module (46 tests)

### DTO Validation (16 tests)

- **Register**: valid data, short password (<8 chars), invalid email, missing fields
- **Login**: valid, missing email, missing password, invalid email format
- **Refresh/Logout**: valid token, missing token, empty string

### Service Integration (13 tests)

| Test | What It Verifies |
|---|---|
| Register (happy path) | Returns access + refresh tokens, user created in DB |

| | |
|---|---|
| Register (duplicate) | Throws "Email already registered" |
| Login (valid) | Returns tokens for correct credentials |
| Login (wrong password) | Throws "Invalid credentials" |
| Login (non-existent) | Throws "Invalid credentials" (same message — no email enumeration) |
| Refresh (rotation) | Issues new tokens, old refresh is invalidated |
| Refresh (reuse detection) | Old token rejected after rotation |
| Refresh (invalid) | Random string throws "Invalid refresh token" |
| Logout | Revokes session, token no longer works |
| Logout (idempotent) | Revoking an already-revoked token doesn't throw |
| Logout all | All sessions for user are invalidated |
| Me (valid) | Returns user profile |
| Me (non-existent) | Throws for deleted user |

### API Integration (17 tests)

| Test | What It Verifies |
|---|---|
| Register (201) | Full HTTP cycle returns tokens |
| Register duplicate (409) | Conflict response |
| Register invalid body (400) | Zod validation on request |
| Login (200) | Returns tokens for valid credentials |
| Login wrong password (401) | "Invalid email or password" |
| Login non-existent (401) | Same error message — no enumeration |
| Refresh (200) | Token rotation via HTTP |
| Refresh reused token (401) | Detects replay attacks |
| Refresh invalid (401) | Random string rejected |
| Logout (204) | Revokes session |
| Logout graceful (204) | No error without refresh token |

| | |
|---|---|
| Logout-all requires auth (401) | Middleware blocks unauthenticated |
| Logout-all (204) | All sessions revoked |
| Me (200) | Returns user profile |
| Me missing token (401) | Requires Authorization header |
| Me invalid token (401) | Expired/malformed rejected |
| Health (200) | Returns `{ status: "ok" }` |

> **Security note:** Login errors use the same message for wrong password and non-existent email. This prevents email enumeration attacks.

## 6.3 Contacts Module (37 tests)

### DTO Validation (11 tests)

- **Create**: valid, minimal (defaults for outlet/topics), missing name, invalid email, empty name, name >255 chars
- **Update**: partial updates, empty object, archivedAt as null (restore), archivedAt as ISO string, invalid email

### Service Integration (13 tests)

| Test | What It Verifies |
|---|---|
| Create | Returns full DTO with score=0 and archivedAt=null |
| Get | Retrieves by ID |
| Get (not found) | Throws "Contact not found" |
| Get (wrong user) | User A can't see User B's contacts |
| List | Paginated results with total count |
| List (excludes archived) | Soft-deleted contacts don't appear |
| List (pagination) | Limit/offset work correctly |
| Update | Partial field updates, unchanged fields preserved |
| Update (not found) | Throws for non-existent |

| | |
|---|---|
| Update (archived) | Throws "Cannot update an archived contact. Restore it first." |
| Delete (soft) | Sets `archived_at`, contact still retrievable |
| Delete (not found) | Throws for non-existent |
| Delete (already archived) | Throws — can't archive twice |

### API Integration (13 tests)

| Test | What It Verifies |
|---|---|
| Create (201) | Full HTTP creation flow |
| Create invalid body (400) | Validation rejects bad data |
| Create unauthenticated (401) | Auth middleware blocks request |
| List (200) | Paginated JSON response |
| List limit/offset | Pagination params work via query string |
| List tenant isolation | User A can't see User B's contacts |
| Get (200) | Single contact by ID |
| Get not found (404) | Proper error response |
| Get invalid ID (400) | Non-numeric ID rejected |
| Update (200) | Partial field update |
| Update not found (404) | Error for non-existent contact |
| Delete (204) | Soft-delete returns no body |
| Delete not found (404) | Error for non-existent contact |

## 6.4 Campaigns Module (38 tests)

### DTO Validation (12 tests)

- **Create**: valid, minimal, empty name, name >255, description >2000
- **Update**: partial, valid statuses (draft/active/paused/completed), invalid status
- **AddContacts**: valid array, empty array, non-positive IDs, non-integers

### Service Integration (14 tests)

| Test | What It Verifies |
|---|---|
| Create | Returns campaign with status="draft" |
| Get / Get (not found) | Standard retrieval + error |
| List / List (excludes archived) | Pagination + soft delete filtering |
| Update | Name + status change |
| Add contacts | Bulk insert via `campaign_contacts` junction |
| Add contacts (duplicate) | `ON CONFLICT DO NOTHING` returns added=0 |
| List participants | Returns contact details via JOIN |
| Remove contact | Deletes from junction table |
| Add contacts (wrong user) | Throws when contacts belong to another user |
| Delete (soft) | Campaign still retrievable but archived |

### API Integration (12 tests)

| Test | What It Verifies |
|---|---|
| Create (201) | Full HTTP creation |
| Create invalid body (400) | Validation rejects |
| Create unauthenticated (401) | Auth required |
| List (200) | Paginated campaigns |
| Get (200) | Single campaign |
| Get not found (404) | Error response |
| Update (200) | Name + status change |
| Delete (204) | Soft-delete |
| Add contacts (201) | Bulk add participants |
| List participants (200) | Contact details via JOIN |
| Remove contact (204) | Delete participant |
| Add contacts empty (400) | Validation rejects empty array |

## 6.5 Interactions Module (28 tests)

### DTO Validation (9 tests)

- Valid directions: `inbound`, `outbound`, `internal`
- Valid channels: `email`, `note`
- Valid statuses: `draft`, `sent`, `delivered`, `failed`, `replied`, `archived`
- Full data with all optional fields (campaignId, subject, occurredAt, metadata)

### Service Integration (10 tests)

| Test | What It Verifies |
|---|---|
| Create outbound | Interaction created + `contact.last_contacted_at` updated |
| Create inbound | Direction and status correct |
| Create note | Internal channel works |
| Create (no contact) | Throws "Contact not found" |
| Create (archived contact) | Throws "Cannot create interaction for an archived contact" |
| Create (score update) | Relationship score recalculated after interaction |
| Get / Get (not found) | Standard retrieval |
| List by contact | Filtered + paginated |
| List all | All user interactions |

### API Integration (9 tests)

| Test | What It Verifies |
|---|---|
| Create (201) | Full HTTP creation with side effects |
| Create invalid direction (400) | Validation rejects bad enum |
| Create unauthenticated (401) | Auth required |
| Create non-existent contact (404) | Contact validation |
| List all (200) | Paginated interactions |
| List filter by contactId | Query param filtering |

| | |
|---|---|
| List filter by campaignId | Query param filtering |
| Get (200) | Single interaction |
| Get not found (404) | Error response |

> **Why score recalculation matters:** The relationship score is computed from interaction history. Creating an outbound email should increase the score. This test verifies the transactional side effect.

## 6.6 Dashboard Module (11 tests)

### Service Integration (5 tests)

| Test | What It Verifies |
|---|---|
| Zero stats | New user gets all zeros, responseRate=0 (not NaN) |
| Counts | totalContacts and activeCampaigns computed from DB |
| Archived excluded | Soft-deleted contacts not counted |
| Recent activity | Audit events mapped to activity feed items |
| Empty activity | No audit events → empty array |

### API Integration (6 tests)

| Test | What It Verifies |
|---|---|
| Stats (200) | Returns dashboard statistics |
| Zero stats for new user | All counters at zero |
| Reflects created contacts | Counts update after inserts |
| Unauthenticated (401) | Auth required |
| Recent activity (200) | Activity feed response |
| Empty for new user | No activity returns empty array |

## 6.7 AI Module (39 tests)

### DTO Validation (9 tests)

- **Generate**: defaults (tone=neutral, length=medium), custom values, invalid tone/length, missing IDs
- **Reject**: optional reason, reason >500 chars

### Prompt Builder (10 tests)

Tests `buildPrompts()` which constructs the system and user prompts sent to OpenAI/Gemini:

- Contact info included in user prompt (name, outlet, topics)
- Campaign objective included
- Recent interactions summarized
- Empty interactions → "No previous interactions"
- Tone instructions (warm → "friendly", direct → "concise")
- Length constraints (short → "under 100 words", long → "200–350 words")
- JSON output format required in system prompt

### Service Integration (8 tests)

AI provider is mocked — no real API calls.

| Test | What It Verifies |
|---|---|
| Generate | Calls provider, stores suggestion in DB, returns subject/body/model |
| Generate (no contact) | Throws "Contact not found" |
| Generate (no campaign) | Throws "Campaign not found" |
| Generate (archived) | Throws "Cannot generate outreach for an archived contact" |
| Accept suggestion | Status transitions to "accepted" |
| Reject suggestion | Status transitions to "rejected" with optional reason |
| Accept (already processed) | Throws — can't accept twice |
| List suggestions | Paginated results |

### API Integration (12 tests)

| Test | What It Verifies |
|---|---|
| Generate (201) | Full HTTP generation flow |
| Generate custom tone/length | Tone and length params forwarded |

| | |
|---|---|
| Generate missing fields (400) | Validation rejects incomplete request |
| Generate unauthenticated (401) | Auth required |
| Generate non-existent contact (404) | Contact validation |
| List suggestions (200) | Paginated suggestions |
| Get suggestion (200) | Single suggestion by ID |
| Get not found (404) | Error for non-existent |
| Accept (200) | Status transition via HTTP |
| Accept already-processed | Rejects double-processing |
| Reject with reason (200) | Status + reason stored |
| Reject without reason (200) | Reason is optional |

## 6.8 Messaging Module (18 tests)

### DTO Validation (6 tests)

- Valid data, optional campaignId/aiSuggestionId, empty subject/body, missing contactId, subject >500

### Service Integration (6 tests)

Both AI provider and email provider are mocked.

| Test | What It Verifies |
|---|---|
| Send | Creates interaction + returns providerMessageId + status="sent" |
| Send (updates contact) | `last_contacted_at` is updated |
| Send (no contact) | Throws "Contact not found" |
| Send (archived) | Throws "Cannot send email to an archived contact" |
| Send (with AI suggestion) | Marks suggestion as "sent" after successful delivery |
| Send (non-accepted suggestion) | Throws "Cannot send: suggestion status is 'draft', expected 'accepted'" |

### API Integration (6 tests)

| Test | What It Verifies |
|---|---|
| Send (201) | Full HTTP send flow |
| Send missing subject (400) | Validation rejects |
| Send unauthenticated (401) | Auth required |
| Send non-existent contact (404) | Contact validation |
| Send archived contact (400) | Archived contact rejected |
| Send with AI suggestion | Marks suggestion as "sent" |

# 7. Frontend Tests — Module Breakdown

## 7.1 API Client (12 tests)

### Core Client (9 tests)

| Test | What It Verifies |
|---|---|
| setTokens | Stores access in memory, refresh in localStorage |
| clearTokens | Removes both tokens |
| loadTokens | Hydrates refresh from localStorage |
| hasRefreshToken | Boolean check for token presence |
| ApiError class | Custom error with status code and message |
| GET with auth | Authorization header attached automatically |
| POST with body | JSON serialization, Content-Type header |
| Error response | Throws `ApiError` with server message |
| 204 No Content | Returns undefined (no body parsing) |

### Refresh & Retry Logic (3 tests)

| Test | What It Verifies |
|---|---|
| 401 → refresh → retry | Automatically retries request with new token after 401 |
| Refresh failure → redirect | Clears tokens and redirects to `/login` |
| Concurrent 401 deduplication | Multiple 401s trigger only one refresh call |

## 7.2 Date Utilities (6 tests)

Uses `vi.useFakeTimers()` to freeze time at a known date:

- `formatDate`: Locale-aware date formatting for two different dates
- `formatRelativeDate`: "seconds ago", "minutes ago", "hours ago", "days ago" — tests four time deltas

## 7.3 Notifications Composable (5 tests)

| Test | What It Verifies |
|---|---|
| notify() | Adds notification with unique ID |
| remove() | Removes by ID |
| Auto-dismiss (success) | Disappears after 4 seconds (fake timers) |
| Auto-dismiss (error) | Disappears after 8 seconds |
| MAX_VISIBLE cap | Queue limits to 5, removes oldest |

## 7.4 Pinia Stores (32 tests)

### Auth Store (7 tests)

| Test | What It Verifies |
|---|---|
| Initial state | user=null, isAuthenticated=false |
| Register | Calls API, stores tokens, sets user |
| Login | Same flow as register |
| Logout | Calls API, clears tokens, resets state |
| tryRestore (no token) | Skips restoration silently |
| tryRestore (valid) | Refreshes tokens, loads user profile |
| tryRestore (expired) | Clears stale tokens, stays logged out |

### Contacts Store (8 tests)

| Test | What It Verifies |
|---|---|
| Initial state | Empty items, total=0, no current |
| fetchList | Populates items and total |
| fetchList error | Sets error string, loading=false |
| fetchOne | Sets current contact |
| create | Calls API, notifies, refreshes list |
| update | Updates current if matching, refreshes |

| | |
|---|---|
| archive | Clears current if matching, refreshes |
| clearCurrent | Resets current to null |

### Campaigns Store (11 tests)

| Test | What It Verifies |
|---|---|
| Initial state | Empty items/participants, no current |
| fetchList | Populates items and total |
| fetchList error | Sets error string |
| fetchOne | Sets current campaign |
| create | Calls API, notifies, refreshes |
| update | Updates current if matching |
| archive | Clears current if matching |
| fetchParticipants | Populates participants array |
| addContacts | Calls API, refreshes participants |
| removeContact | Calls API, refreshes participants |
| clearCurrent | Resets current and participants |

### Interactions Store (6 tests)

| Test | What It Verifies |
|---|---|
| Initial state | Empty items, total=0 |
| fetchList with filter | Passes contactId param |
| fetchList default | Uses empty params |
| fetchList error | Sets error string |
| create | Calls API, notifies |
| clearList | Resets items and total |

## 7.5 Router Guards (6 tests)

| Test | What It Verifies |
|---|---|
| Protected → redirect | Unauthenticated → `/login?redirect=/contacts` |
| Root → redirect | Unauthenticated → `/login?redirect=/` |
| Guest allowed | Unauthenticated can visit `/login` |
| Guest → dashboard | Authenticated on guest route → redirected to `/` |
| Protected allowed | Authenticated can visit protected routes |
| tryRestore once | Called only on first navigation |

## 7.6 View Components (24 tests)

### LoginView (7 tests)

| Test | What It Verifies |
|---|---|
| Renders heading | "Sign in" title visible |
| Renders inputs | Email and password fields present |
| Link to register | RouterLink to `/register` |
| Submit success | Calls auth.login, navigates to `/` |
| ApiError display | Shows "Invalid email or password" |
| Generic error | Shows "Login failed" for non-API errors |
| Button disabled | Shows "Signing in..." while submitting |

### RegisterView (7 tests)

| Test | What It Verifies |
|---|---|
| Renders heading | "Create your account" title |
| Renders inputs | Email + 2 password fields |
| Password mismatch | Client-side validation "Passwords do not match" |
| Password too short | "at least 8 characters" |
| Submit success | Calls auth.register, navigates to `/` |

| | |
|---|---|
| API error display | Shows server error message |
| Link to login | RouterLink to `/login` |

### OutreachDraftView (10 tests)

| Test | What It Verifies |
|---|---|
| Renders heading | "AI Outreach Draft" title |
| Dropdowns populated | Contact and campaign options visible |
| Tone/length options | Warm/Neutral/Direct, Short/Medium/Long |
| Generate disabled | Button disabled without selections |
| Error without selections | Validation message shown |
| Successful generation | Draft displayed with subject, body, model |
| Accept/reject buttons | Shown after generation |
| Send button | Appears after accepting draft |
| Generation error | Error message displayed |
| Suggestion history | Recent suggestions list populated |

## 7.7 Form Components (25 tests)

### ContactFormModal (8 tests)

| Test | What It Verifies |
|---|---|
| New title | "New Contact" when no contact prop |
| Edit title | "Edit Contact" with contact prop |
| Pre-fill | Fields populated in edit mode |
| Create submit | Calls store.create with form data |
| Update submit | Calls store.update with contact ID |
| Emit saved/close | Events emitted on success |
| Error display | Shows API error message |

| | |
|---|---|
| Cancel | Emits close event |

### CampaignFormModal (8 tests)

| Test | What It Verifies |
|---|---|
| New title | "New Campaign" without campaign prop |
| Edit title | "Edit Campaign" with campaign prop |
| Status dropdown | Only visible in edit mode |
| Create submit | Calls store.create with name/description/objective |
| Update submit | Calls store.update with status |
| Emit saved/close | Events emitted on success |
| Error display | Shows API error message |
| Cancel | Emits close event |

### InteractionForm (9 tests)

| Test | What It Verifies |
|---|---|
| Title | "Log Interaction" heading |
| Contact dropdown | Visible when no contactId prop |
| Hidden dropdown | Hidden when contactId provided |
| Direction radios | outbound/inbound/internal buttons |
| Channel radios | email/note buttons |
| Create submit | Calls store.create with full payload |
| Emit saved/close | Events emitted on success |
| Error display | Shows error message |
| Cancel | Emits close event |

# 8. E2E Smoke Tests (12 tests)

E2E tests run against real servers (backend + frontend + PostgreSQL) using Playwright with Chromium. Each test creates a fresh user to avoid data collision.

## 8.1 Auth Journey (5 tests)

| Test | What It Verifies |
|---|---|
| Register | Create account → land on dashboard |
| Login | Register → logout → login → dashboard |
| Redirect | Unauthenticated `/contacts` → `/login` |
| Wrong credentials | "Invalid email or password" error shown |
| Logout | Session cleared, protected routes blocked |

## 8.2 Contacts Journey (2 tests)

| Test | What It Verifies |
|---|---|
| Create + list | Open modal → fill form → submit → contact appears in list |
| View detail | Click contact → navigate to `/contacts/:id` → name visible |

## 8.3 Campaigns Journey (2 tests)

| Test | What It Verifies |
|---|---|
| Create + list | Open modal → fill form → submit → campaign appears in list |
| View detail | Click campaign → navigate to `/campaigns/:id` |

## 8.4 Outreach Journey (3 tests)

| Test | What It Verifies |
|---|---|
| Page loads | Heading, dropdowns populated with seeded data |
| Generate disabled | Button disabled without selections |

| | |
|---|---|
| Select + enable | Selecting contact and campaign enables generate button |

---

## 9. CI/CD Pipeline

The GitHub Actions workflow ( `.github/workflows/ci.yml` ) runs on every push to `main` / `dev` and on PRs targeting those branches.

### 9.1 Job Architecture

```
build-shared ──┬── test-backend ──┐
               ├── test-frontend ──┼── test-e2e (main only)
               └── build ──────────┘
```

| Job | Runs On | What It Does |
|---|---|---|
| `build-shared` | Every push/PR | Builds `shared-types` , uploads artifact |
| `test-backend` | Every push/PR | Spins up PostgreSQL 15, runs migrations, runs 254 tests |
| `test-frontend` | Every push/PR | Runs 110 unit + component tests |
| `build` | Every push/PR | TypeScript compilation check (backend + frontend) |
| `test-e2e` | main branch only | Installs Chromium, runs 12 Playwright smoke tests |

### 9.2 Key Design Decisions

- **`build-shared` as a dependency gate:** All jobs download the pre-built `shared-types` artifact. This avoids building it 4 times.
- **PostgreSQL as a service container:** `postgres:15-alpine` runs alongside the test runner with health checks. No Docker-in-Docker.
- **Concurrency control:** `cancel-in-progress: true` kills stale runs when a new push arrives.
- **E2E gated to main:** E2E tests are slow (~20s) and depend on both servers. Running them only on main/PRs-to-main keeps dev branch CI fast.
- **Artifact upload on failure:** Playwright screenshots and traces are uploaded even when tests fail, enabling debugging without reproducing locally.

# 10. Challenges & Solutions

## 10.1 `happy-dom` EPERM on Windows

**Problem:** `happy-dom` caused persistent `EPERM` file locking errors on Windows. The `.pnpm-store` couldn't delete or overwrite its files.

**Solution:** Added a pnpm override in the root `package.json`:

```
{ "pnpm": { "overrides": { "happy-dom": "npm:@pnpm/empty-pkg@1.0.0" } } }
```

Then used `jsdom` instead.

## 10.2 `instanceof` Broken for AppError Subclasses

**Problem:** `AppError` uses `Object.setPrototypeOf(this, AppError.prototype)` for ES5 compatibility. This breaks `instanceof` checks under Vitest's module isolation.

**Solution:** Assert on error messages instead of types. More readable anyway.

## 10.3 Database Deadlocks in Parallel Test Files

**Problem:** Integration test files running in parallel deadlocked on `TRUNCATE CASCADE`.

**Solution:** `fileParallelism: false` in `vitest.config.ts`. Total suite time: ~37s.

## 10.4 `vitest.config.ts` Outside `rootDir`

**Problem:** TypeScript complained that `vitest.config.ts` was outside the `src/` rootDir.

**Solution:** Added `"exclude": ["vitest.config.ts"]` to `backend/tsconfig.json`.

## 10.5 Setup File Double-Path

**Problem:** With `root: "src"` and `setupFiles: ["./src/__tests__/setup.ts"]`, Vitest resolved to `src/src/__tests__/setup.ts`.

**Solution:** Changed to `./__tests__/setup.ts` (relative to root, which is already `src/`).

## 10.6 Rate Limiting in E2E Tests

**Problem:** Auth rate limiter (20 req/15 min) blocked E2E test registrations after multiple runs.

**Solution:** Rate limit raised to 200 in development mode:

```
const authLimiter = rateLimit({
  windowMs: 15 * 60 * 1000,
  max: process.env.NODE_ENV === "development" ? 200 : 20,
});
```

### 10.7 Vue `watch` Not Triggering on Mount

**Problem:** Modal components using `watch(() => props.open)` didn't trigger when mounted with `open: true` because the watcher only fires on *change*.

**Solution:** Mount with `open: false`, then call `wrapper.setProps({ open: true })` to trigger the watcher. Also stub `HTMLDialogElement.prototype.showModal` since jsdom doesn't implement `<dialog>`.

---

## 11. Running the Tests

```
# — Backend (254 tests) —
cd backend && pnpm test              # all tests
cd backend && pnpm test:watch        # watch mode
cd backend && pnpm test:coverage     # with coverage

# — Frontend (110 tests) —
cd frontend && pnpm test             # all tests
cd frontend && pnpm test:watch       # watch mode

# — E2E (12 tests) — requires both servers running —
pnpm test:e2e                        # headless Chromium
pnpm test:e2e:headed                 # visible browser

# — Everything from root —
pnpm test                            # backend + frontend (sequential)
pnpm test:e2e                        # E2E (auto-starts servers)
```

**Prerequisites:**

- PostgreSQL running on port 5433 (via Docker: `docker compose up -d` in `backend/`)
- Test database `pr_ai_db_test` must exist (for backend tests)
- Development database `pr_ai_db` must exist (for E2E tests)

- `backend/.env.test` configured with test DB connection
- `shared-types` built: `pnpm --filter @pr-outreach/shared-types build`

---

## 12. Test Coverage Summary

| Module | DTO | Service | API | Total |
|---|---|---|---|---|
| Shared (errors, utils, middleware) | — | — | — | **37** |
| Auth | 16 | 13 | 17 | **46** |
| Contacts | 11 | 13 | 13 | **37** |
| Campaigns | 12 | 14 | 12 | **38** |
| Interactions | 9 | 10 | 9 | **28** |
| Dashboard | — | 5 | 6 | **11** |
| AI | 9+10 | 8 | 12 | **39** |
| Messaging | 6 | 6 | 6 | **18** |
| **Backend subtotal** | **73** | **69** | **75** | **254** |

| Module | Unit | Component | Total |
|---|---|---|---|
| API Client | 12 | — | **12** |
| Date Utils | 6 | — | **6** |
| Notifications | 5 | — | **5** |
| Stores (auth, contacts, campaigns, interactions) | 32 | — | **32** |
| Router Guards | 6 | — | **6** |
| Views (Login, Register, Outreach) | — | 24 | **24** |
| Form Modals (Contact, Campaign, Interaction) | — | 25 | **25** |
| **Frontend subtotal** | **61** | **49** | **110** |

| Layer | Tests | Files |
|---|---|---|
| Backend (unit + integration + API) | 254 | 26 |

| | | |
|---|---|---|
| Frontend (unit + component) | 110 | 15 |
| E2E (Playwright) | 12 | 4 |
| **Grand total** | **376** | **45** |

---

## 13. Implementation Phases — Complete

| Phase | Focus | Tests | Status |
|---|---|---|---|
| Phase 1 | Foundation + Auth | 37 shared + 29 auth + 27 frontend foundation | ✅ Done |
| Phase 2 | All backend services + DTOs | 116 service + DTO tests | ✅ Done |
| Phase 3 | API integration (supertest) | 75 HTTP integration tests | ✅ Done |
| Phase 4 | Frontend stores + router + client | 53 store/router/client tests | ✅ Done |
| Phase 5 | Frontend component tests | 49 component tests | ✅ Done |
| Phase 6 | E2E smoke tests (Playwright) | 12 E2E tests | ✅ Done |
| Phase 7 | CI/CD pipeline (GitHub Actions) | 5-job workflow | ✅ Done |

---

*Built with care across 7 phases. Break things in tests, not in production.*